Assignment 1

Fine Detail and the Big Picture

Overview

In this assignment you build the basic tools of image processing yourselves, and then put them to work. You will make images that change identity depending on how far away you stand, blend an apple and an orange so that no one can tell where the apple ends, and shrink photos without squashing the people in them. Along the way, you will see in practice why blurring before you differentiate or downsample matters so much.

The parts build on each other. The convolution of 1.1 runs every filter that follows, and the gradients of 1.2 return in Part 3 to decide which pixels of a photo matter. There are many small sections, so start early and finish 1.1 first. Once Part 1 is done, Parts 2 and 3 can be worked on side by side.

Before You Start

Teams and language. Work in teams of three, in Python.

Starter pack. Download it first. It holds the images named in the sections, and two small helpers: `utils.py` to load, save and show images, and `align_images.py` to align two images by two pairs of matching points.

Images. Use medium-size images, no more than about 1000 pixels on the longer side, so your code runs in reasonable time. Grayscale is enough everywhere, and color is welcome: apply your convolution to each channel. Derivatives, DoG filters, high-pass images and Laplacian levels go negative: show them with zero as mid-gray, since `utils.show` clips to $[0, 1]$.

What you write yourselves. The core of every section is yours to write: convolution, filters, pyramids, blending and seam carving. From 1.2 onward, run every blur and derivative on the separable version of your own convolution. Library functions are fine only to load, save and show images, to convert their color or bring them to a working size, to align them with `align_images.py`, to draw masks, to take the Fourier transform, to resize an image for the comparison in 3.2, and to check your results.

Online resources and AI tools. You may use them to help, but every team member must be able to explain any part of what you submit.

Part 1: Filters and Edges

This part builds intuition for 2D convolution and filtering, and uses them to find edges.

1.1 Convolution from Scratch

Every filter in this assignment, from the derivatives that detect edges to the Gaussians that build pyramids, is a convolution. In this section you implement it yourselves, efficiently enough to serve the rest of the assignment.

Recall that convolving with a filter is correlating with the filter flipped along both axes.

Implement 2D convolution for grayscale (single-channel) images in two versions, each padding the input with zeros so that the output has the same size as the input.

- A loop version iterates over the pixels and the filter entries.
- A vectorized version has no loop over the pixels, only whole-image NumPy operations.

Many filters, including the box and the Gaussian, are separable: they are the product of a column and a row. Add a third version that exploits this.

- Test whether the filter has rank 1 up to rounding error, using its SVD.
- If so, convolve with the column, then with the row.
- Otherwise, use the vectorized version.

Finally, run a sanity check on small random images, with random filters of odd heights and widths, both separable and not.

- All three versions must match `scipy.signal.convolve2d` in `mode="same"` to rounding error.
- Time them on a 256×256 image with Gaussian filters from 3×3 to 31×31, report the times side by side, and explain the differences.

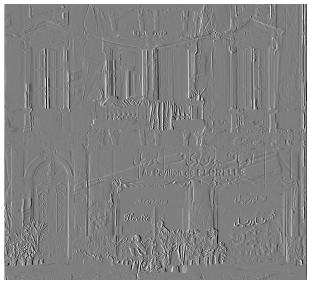
1.2 Finite Difference Operator

An edge is a sharp change in intensity, and derivatives measure change. Finite differences are the simplest estimate of an image's derivatives, and already a first edge detector.

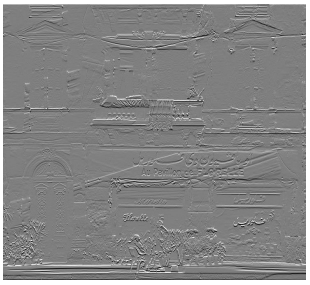
With your convolution from 1.1, filter a grayscale image, from the starter pack or of your own, with $D_x = [1 \;\; 0 \;\; -1]$ and $D_y = D_x^\mathsf{T}$, the finite differences along x and y.

- Combine the two derivatives into the gradient magnitude $\sqrt{(\partial I/\partial x)^2 + (\partial I/\partial y)^2}$.
- Binarize the magnitude with a threshold to obtain an edge image.

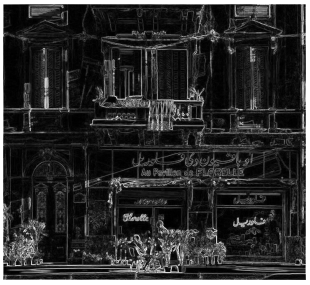
Show the four maps: the two derivatives, the gradient magnitude and the edge image.



$\partial I/\partial x$



$\partial I/\partial y$



Gradient magnitude



Edges, corner enlarged

Justify your threshold, and explain where the lines along the image frame come from.

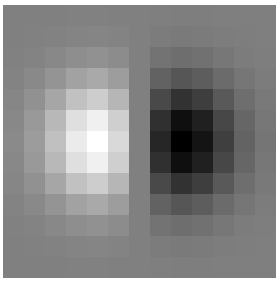
1.3 Derivative of Gaussian (DoG) Filter

Derivatives amplify noise, which is why the edges of 1.2 are noisy. The remedy is to blur before differentiating, and since convolution is associative, the blur and the derivative combine into a single filter: the derivative of Gaussian (DoG).

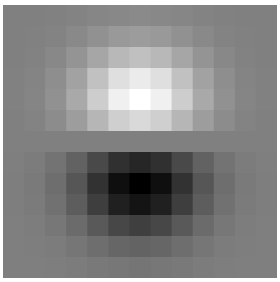
Build a second version of 1.2 on the same image, with the blur folded into the filters.

- Build a 2D Gaussian kernel, and filter the kernel itself with D_x and with D_y: the two results are the DoG filters.
- Apply each DoG filter to your image with a single convolution.
- Repeat the steps of 1.2 on the two outputs: the gradient magnitude, then the edge image.

Show the two DoG filters as images, and the four maps of 1.2 for the new filters.



DoG in x, $\sigma = 2$



DoG in y, $\sigma = 2$



Gradient magnitude



Edges

State your σ and threshold, and explain how and why the maps differ from those of 1.2.

Part 2: Frequencies, Pyramids and Blending

A blur keeps an image's low frequencies, and the image minus its blur keeps the high ones. This part builds on that split: first to mix two images, then to understand sampling, and finally to decompose and blend images at every scale.

From here on, pad by reflection, with `np.pad(..., mode="reflect")`: zeros would darken the border of every blur.

2.1 Hybrid Images

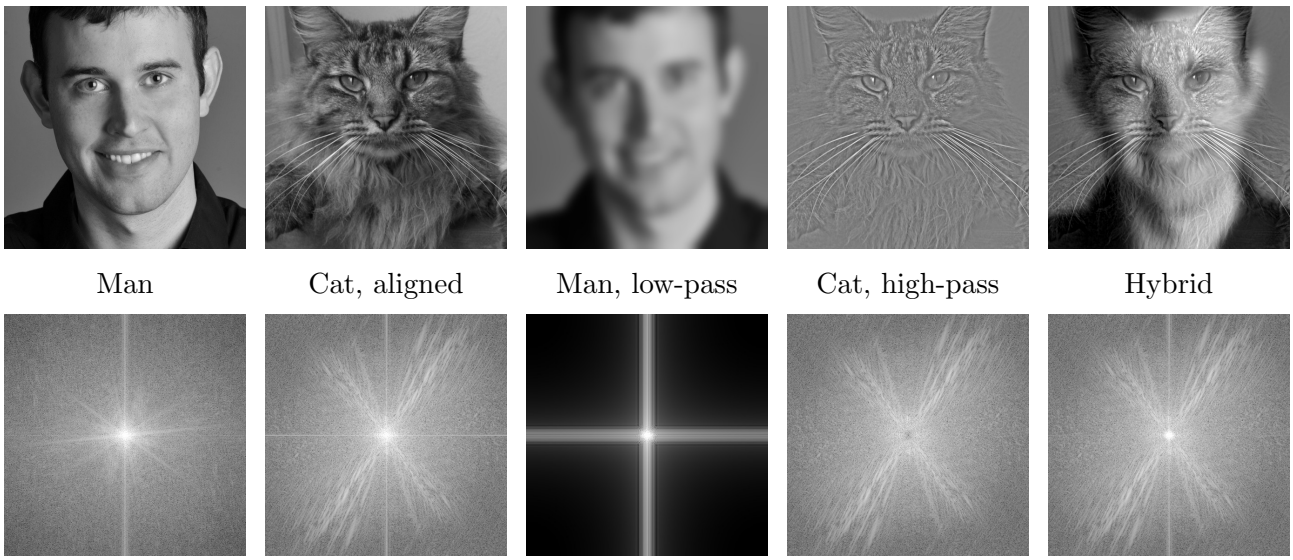
Hybrid images change interpretation with viewing distance (Oliva, Torralba & Schyns, 2006): up close the high frequencies dominate, and from far away only the low ones remain. A hybrid adds the high frequencies of one image to the low frequencies of another.

Make three hybrid images: the man-and-cat pair from the starter pack, and two pairs of your own. One of them may fail, as long as you explain why. For each pair:

- Align the two images with `align_images.py` from the starter pack. Badly aligned images just look like a double exposure.
- Low-pass the first image with a Gaussian.
- High-pass the second: the image minus its blur.
- Add the two, and tune the two σ values, which set the cutoff frequencies.

For your favorite hybrid, show the whole process, including the log magnitude of the Fourier transform of the two inputs, of the two filtered images and of the hybrid, computed on the grayscale images with

```
np.log(np.abs(np.fft.fftshift(np.fft.fft2(gray_image))))
```



Man | Cat, aligned | Man, low-pass | Cat, high-pass | Hybrid

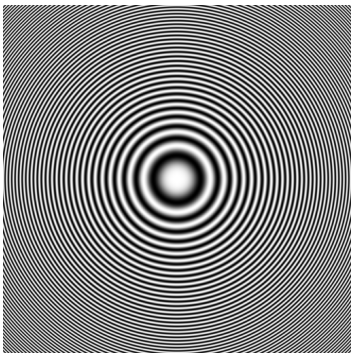
Log magnitude of the Fourier transform of each image above

Report the two σ values you chose for each hybrid.

2.2 Aliasing

Shrinking an image samples it on a coarser grid, and fine detail does not always survive the move. This section shows what happens to that detail, and what it takes to shrink an image without creating false patterns.

Take `rings.png` from the starter pack, an image packed with fine detail. View it at 100% zoom: any viewer that shrinks it on screen is downsampling it too, and shows false circles. That is the very effect you are about to study.



Shrink it to a quarter of its width and height in two ways, and show the two results side by side.

- Keep every fourth pixel in each direction.
- Blur it with a Gaussian first, then keep every fourth pixel.

State the smallest σ at which you no longer see false circles. Explain what happened in each result and why, and what this means for how a Gaussian pyramid should be built.

2.3 Gaussian and Laplacian Pyramids

A Gaussian pyramid stores an image at halved sizes, each level a blurred copy of the one before. A Laplacian pyramid stores the detail lost between levels, so the image can be rebuilt.

Write two functions first, both blurring with $\frac{1}{16}[1 \;\; 4 \;\; 6 \;\; 4 \;\; 1]$ along x and y.

- The first, `reduce`, blurs the image, then keeps every other row and column.
- The second, `expand`, places each pixel at the top left of a 2×2 block of zeros, blurs, and multiplies by 4, since three pixels in four are zeros.

Then build both pyramids for an image of your choice whose sides are not powers of 2.

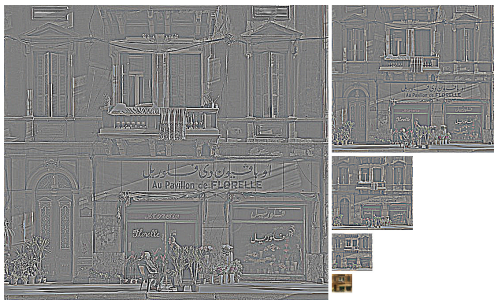
- For the Gaussian pyramid, apply `reduce` repeatedly.
- For the Laplacian pyramid, subtract from each Gaussian level the next level expanded to its size, and end with the smallest Gaussian level itself.

To rebuild, start from the smallest level and repeat: expand, then add the Laplacian level of that size. When a side is odd, expanding gives one row or column too many: drop the last one. Write everything yourselves: no `cv2.pyrDown`, `cv2.pyrUp` or `skimage` pyramid functions.

Show both pyramids and the rebuilt image with its error.



Gaussian pyramid



Laplacian pyramid, contrast boosted

Explain why the rebuild is exact, while Lecture 3's version, built from each level minus its blur, is not. Give the Gaussian pyramid's total size relative to the image, and explain the number.

2.4 Multiresolution Blending

Pasting one image into another along a sharp cut leaves a visible seam. Multiresolution blending (Burt & Adelson, 1983) hides it by blending each frequency band over its own width: coarse structure over a wide region, fine detail over a narrow one. Pyramids give these widths for free.

Blend two images along a mask: an image of their size, white (1) where you want the first image and black (0) where you want the second.

- Build the Laplacian pyramids of the two images, and the Gaussian pyramid of the mask.
- At each level, combine the Laplacian levels L_1 and L_2 of the two images as $ML_1 + (1-M)L_2$, with M the mask's level of that size.
- Rebuild the result from the combined pyramid, as in 2.3.

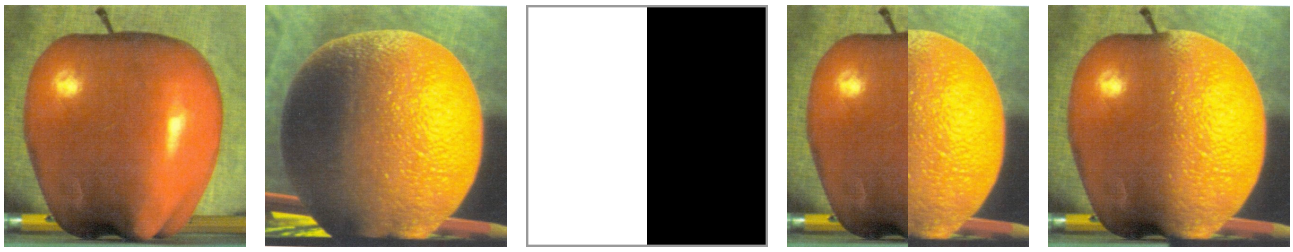
Keep the mask sharp: its Gaussian pyramid softens it for you, a little at the fine levels and a lot at the coarse ones, and that is what hides the seam. Choose the number of levels for each pair: too few leave the seam visible, and too many wash out a small pasted object.

Make three blends.

- The oraple uses `apple.jpg` and `orange.jpg`, with the left half of the mask white.
- The other two use pairs of your own, such as a face on a fruit, each with an irregular mask.

Place one image onto the other with `align_images.py` and two points on each, as in 2.1. Draw each mask in code, with `cv2.ellipse` or `cv2.fillPoly`, or paint it in any image editor and load it. The hand and eye in the starter pack are there to practice on, and do not count.

Show each blend with its two inputs, its mask and a hard cut. For your favorite, also show, level by level, ML_1, $(1 - M)L_2$ and their sum.



Apple | Orange | Mask | Hard cut | Blend

Say how you made each mask and how many levels you used, and explain why blending band by band hides the seam that the hard cut shows.

Part 3: Seam Carving

Resizing a photo to a new aspect ratio leaves two bad options: squash everything, or crop things out. Seam carving (Avidan & Shamir, 2007) removes the pixels that matter least instead, so the important content keeps its shape. The lectures do not cover it: read Section 3 of the paper, and Avik Das's post for a gentle walk through its dynamic program.

3.1 Finding and Removing Seams

To narrow an image by one column without squashing it, take one pixel out of every row, where the image has the least to lose. Those pixels form a seam, and removing seam after seam narrows the image as far as you need.

Measure how much each pixel matters with an energy: edges matter, flat regions like sky do not.

- Use the gradient magnitude of 1.2 as the energy, computed with your own filters.

Then remove the seam with the lowest total energy, again and again.

- A seam takes one pixel per row, each touching the one above it, directly or diagonally.
- Find the cheapest seam by dynamic programming: compute the minimum cumulative energy of a seam ending at each pixel, row by row, then trace the seam back from the bottom row.
- Remove it from all channels, recompute the energy, and repeat to the width you want.
- Alongside the image, keep an array holding each pixel's original column, and remove the same seams from it, so you can draw each seam where it sat in the original.
- For horizontal seams, transpose the image's rows and columns.

Show the energy of one image and the seams you removed, drawn on the original.



Energy



Removed seams (30% of the width), in red

Photo (3.1, 3.2, Bonus) adapted from Broadway Tower by Newton2, cropped by Yummifruitbat, CC BY 2.5.

Explain how your dynamic program finds the cheapest seam, and why trying every possible seam instead would take forever.

3.2 Results and Comparisons

Seam carving is worth its cost only where it beats the simple options. This section compares it with normal resizing and cropping, and looks for where it breaks.

Reduce the width of `rocket.jpg`, `home.jpg` and `starry_night.jpg` from the starter pack, and of three of your own, by about 30%. At least one of yours should show a person: any photo of anyone works.

- Compare seam carving with normal resizing and with a center crop, at the same width.
- Try at least two energy functions, such as the finite differences of 1.2 and the DoG of 1.3, and see which seams each one picks.
- Find at least one image where seam carving fails badly.

For each image, show the original and its three narrower versions side by side. For one image, also show the seams that each energy function picks.



Original



Resized



Cropped



Seam carved

Explain how the seams of the two energy functions differ, and what makes seam carving fail on your failure case.

Bonus

Seams can also make an object disappear. Inserted instead of removed, they widen an image back to its size. Section 4.6 of the paper covers the first, Section 4.3 the second, and its Figure 12 shows both together.

First, remove an object from a photo of your own.

- Paint a mask over the object, the same size as the photo, as in 2.4.
- Compute the energy in `float64`, and set it to a large negative value, such as -10^6, on the masked pixels, so that the cheapest seam passes through them.
- Remove that seam from the photo and from the mask, and repeat until no masked pixel is left. Call the number of seams removed k.

Then grow the reduced photo back by k seams.

- Make a copy of the reduced photo, and remove k seams from the copy, as in 3.1. The copy is only used to find them.
- Alongside the copy, keep an array holding each pixel's column in the reduced photo, and remove the same seams from it, so you know where each seam sat in the reduced photo.
- Widen the reduced photo row by row: write each pixel once, but each pixel of the k seams twice, the first copy averaged with its left neighbor and the second with its right neighbor. At the border, use the pixel itself as the missing neighbor.

Show the photo with its mask, the photo without the object, and the photo back at its original width, with the inserted seams drawn.



Mask over the walker, in red



Walker gone, width restored, new seams in green

Explain why the k seams come from the copy, not from inserting the cheapest seam k times.

Submission and Deliverables

What to submit. One submission per team: your code, and a single PDF report with the names and IDs of the three members on its first page.

The report. Walk us through your results part by part. Show every image asked for, explain what you did, and answer every question along the way. Include your 1.1 convolution as a code snippet, and give the Bonus, if you attempt it, a section of its own. Images speak louder than words, so lean on them.